

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

**Programa de Licenciatura en Ingeniería en
Computadores**

Curso: CE-4301 Arquitectura de Computadores I



Especificación Proyecto Individual

Profesor:

Dr.-Ing. Jeferson González Gómez

**Fecha de entrega: Viernes 4 de Septiembre de 2026
11:59 p.m, tecDigital**

Proyecto Individual: Codificador Educativo de Instrucciones RISC-V

1. Objetivo

El estudiante implementará una herramienta que traduzca una única instrucción del subconjunto RISC-V RV32I definido en este documento a su codificación binaria de 32 bits, mostrando de forma visual el significado de cada campo del formato correspondiente (R, I, S o B). El estudiante deberá además validar el correcto funcionamiento de su herramienta comparando su salida contra la de un ensamblador/*disassembler* oficial de RISC-V de 32 bits.

Modalidad: Individual.

Duración: 2 semanas.

2. Descripción General

Comprender cómo una instrucción en lenguaje ensamblador se traduce a una secuencia de bits es uno de los aprendizajes fundamentales de la arquitectura de computadores: cada campo de la instrucción (*opcode*, registros, *funct3/funct7*, inmediato) tiene un significado preciso que determina cómo el procesador la decodifica y ejecuta.

En este proyecto el estudiante construirá, sobre un subconjunto reducido de instrucciones RV32I, un modelo propio de codificación que reciba **una instrucción a la vez** en formato texto (mnemónico y operandos, tal como se escribiría en ensamblador) y produzca su codificación en binario, junto con una explicación del rol y valor de cada campo. No se requiere traducir programas completos ni resolver saltos relativos a etiquetas (*labels*): el alcance se limita deliberadamente a la codificación de instrucciones individuales ya con sus operandos numéricos resueltos.

3. Especificación

3.1. Subconjunto de instrucciones

El modelo deberá soportar **únicamente** las siguientes 12 instrucciones de RV32I, que cubren los cuatro formatos de interés (R, I aritmético, I de carga, S y B):

Categoría	Formato	Instrucciones
Aritmética registro-registro	R	add, sub, and, or
Aritmética con inmediato	I	addi, andi
Carga desde memoria	I	lw, lb
Almacenamiento en memoria	S	sw, sb
Salto condicional	B	beq, bne

Los valores exactos de *opcode*, *funct3* y *funct7* (según aplique) para cada una de estas instrucciones **no se proveen** en este documento: es responsabilidad del estudiante consultarlos en el manual oficial de la ISA RISC-V [1] y documentar la fuente utilizada.

3.2. Modo de operación

La herramienta deberá operar recibiendo **una única instrucción por ejecución**, pasada como **un solo argumento de línea de comandos**, con los operandos ya resueltos a valores numéricos.

Punto de entrada fijo: Independientemente del lenguaje o la estructura interna de su solución, el repositorio deberá incluir en su raíz un script ejecutable llamado `run.sh` que sirva como único punto de entrada, invocado exactamente así:

```
1 ./run.sh "<instruccion>"
```

Por ejemplo:

```
1 ./run.sh "add x5, x6, x7"
2 ./run.sh "addi x10, x1, -12"
3 ./run.sh "lw x5, 8(x6)"
4 ./run.sh "sw x8, -4(x2)"
5 ./run.sh "beq x1, x2, 8"
```

`run.sh` es responsable de invocar internamente lo que sea necesario (intérprete de Python, binario compilado, activación de un entorno virtual, etc.). Este punto de entrada único es indispensable porque tanto el estudiante como el profesor validarán la herramienta de forma automatizada (ver 3.4) ejecutando **siempre el mismo comando**, sin importar cómo esté implementada la solución de cada estudiante. Se proporciona, como parte del kit del proyecto (`kit/encoder_skeleton.py` y `kit/run.sh`), un esqueleto en Python junto con su `run.sh` correspondiente que ya implementan este contrato de entrada/salida; su uso es opcional, pero si se implementa en otro lenguaje o desde cero, debe respetarse el mismo punto de entrada y contrato de salida.

Para cada instrucción de entrada, la salida deberá incluir, como mínimo:

- El formato identificado (R, I, S o B).
- La codificación completa en binario (32 bits) y en hexadecimal.
- Una representación visual de los 32 bits divididos en sus campos (p. ej. `opcode`, `rd`, `funct3`, `rs1`, `rs2`, `funct7`, o el inmediato ensamblado según el formato), indicando el rango de bits y el valor decimal/binario de cada campo.
- Una breve explicación textual del rol de cada campo en esa instrucción específica (p. ej. qué registro fuente/destino representa, cómo se interpreta el inmediato, etc.).

El formato visual de los campos (colores, tabla, arte ASCII, etc.) queda a criterio del estudiante, siempre que sea claro y permita identificar cada campo y su valor sin ambigüedad.

Salida procesable para validación automática: Adicionalmente a la salida explicativa, la herramienta deberá imprimir, en alguna línea de su salida estándar, la codificación en el formato exacto `HEX: 0XXXXXXXX` (8 dígitos hexadecimales, con el prefijo `0x`), para permitir su verificación automatizada. Esta línea puede convivir con el resto de la salida explicativa.

3.3. Validación contra herramientas oficiales

El estudiante deberá instalar por su cuenta un toolchain de RISC-V de **32 bits (rv32)** que incluya un ensamblador y `objdump` capaz de desensamblar código de 32 bits. La investigación de cómo obtener/installar dicho toolchain es parte del alcance del proyecto y no se proveerá una guía.

Para validar la herramienta, el estudiante deberá, para cada una de las 12 instrucciones soportadas, construir **al menos 3 casos de prueba distintos** (no solo variando los registros), cubriendo diferentes escenarios según aplique a cada instrucción: valores positivos, valores negativos, y valores límite (p.ej. el máximo/mínimo inmediato representable, registro `x0`, o un salto de desplazamiento cero). Cada caso deberá ensamblarse con el toolchain oficial, obtener la codificación de referencia con `objdump -d`, y compararse contra la salida de su propio modelo. Se deberá documentar esta comparación (instrucción, salida del modelo, salida de `objdump`, coincidencia o no) para los al menos 36 casos resultantes (12 instrucciones $\times$ 3 casos).

3.4. Verificación automática

Además de la validación manual descrita arriba, el profesor evaluará la herramienta de cada estudiante ejecutando un **script de verificación automática** que corre 36 escenarios (12 instrucciones $\times$ 3 casos: positivo, negativo, límite) contra el toolchain oficial y contra la herramienta del estudiante, invocándola **siempre como** `./run.sh «instruccion»`. Este script es de uso exclusivo del profesor y no se distribuirá.

Para que el estudiante pueda autoevaluarse desde el primer día sin necesidad de dicho script, se incluye en el kit el archivo `kit/vectores_ejemplo.txt`: un conjunto de instrucciones de ejemplo junto con su codificación correcta (formato `instruccion ; 0xHEX`), una por línea. Estos vectores son **distintos** de los que usará el profesor en la verificación final, por lo que no deben usarse como sustituto de los 36 casos propios que exige la validación manual de la sección anterior; su único propósito es permitir una comprobación rápida y objetiva de que la herramienta funciona correctamente antes de construir esos casos propios.

Como el punto de entrada (`run.sh`) y el contrato de salida (`HEX: 0x...`) son fijos e iguales para todos los estudiantes, el profesor podrá ejecutar la verificación automática de cualquier entrega sin necesidad de leer instrucciones particulares de invocación. El README deberá documentar, de forma breve y precisa, únicamente la **preparación previa** que requiera el entorno (p.ej. instalar Python, un intérprete, o dependencias vía `pip install -r requirements.txt`) para que `run.sh` funcione correctamente una vez ejecutada; no debe proponerse un comando de invocación alternativo. Los valores concretos de registros e inmediatos que use el profesor en la verificación automática pueden diferir de los que el estudiante haya elegido en su propia validación manual: la herramienta debe generalizar correctamente a cualquier instrucción del subconjunto soportado, no solo a los casos específicos documentados.

3.5. Entregables

1. Código fuente del modelo de codificación (se recomienda Python, aunque puede usarse otro lenguaje con justificación).

2. Evidencia de la validación de los al menos 36 casos de prueba (12 instrucciones $\times$ 3 escenarios) contra el toolchain oficial (tabla o log de comparación).
3. Documentación técnica en formato *Markdown* (.md) que incluya: instrucciones soportadas y cómo se obtuvieron sus campos de codificación, arquitectura del código, ejemplos de la salida explicativa para al menos 4 instrucciones distintas (una por formato: R, I, S, B), evidencia de la comparación contra la herramienta oficial, instrucciones de instalación del toolchain que utilizó (breve, sin necesidad de tutorial extenso), e instrucciones de instalación/preparación de su propia herramienta (dependencias y entorno, necesarias para que `./run.sh «instruccion»` funcione correctamente en la verificación automática de la sección 3.4).

4. Evaluación del proyecto

La evaluación del proyecto se da bajo los siguientes rubros:

- **Funcionalidad (70 %)**: El profesor evaluará de manera independiente la herramienta, ejecutándola con instrucciones de su elección dentro del subconjunto soportado.

Los aspectos que se revisarán **durante** la evaluación funcional son los siguientes:

1. Codificación correcta de las 12 instrucciones soportadas, para distintos valores de registros e inmediatos (incluyendo inmediatos negativos donde aplique).
 2. Correcta identificación y desglose visual de los campos según el formato (R, I, S, B).
 3. Explicación textual correcta del significado de cada campo.
 4. Validación exitosa contra el toolchain oficial de 32 bits para los 36 casos requeridos, verificada mediante el script de verificación automática (sección 3.4).
- **Documentación de la implementación (30 %)**: La documentación en formato *Markdown* (.md) deberá contener al menos la siguiente información:
 1. Descripción de la arquitectura del código y decisiones de diseño.
 2. Fuente(s) consultada(s) para los campos de codificación de cada instrucción.
 3. Ejemplos de salida explicativa (uno por formato: R, I, S, B).
 4. Evidencia de la validación contra el toolchain oficial.
 5. Instrucciones de instalación del toolchain, y de instalación/preparación y uso de la herramienta.

Referencias

- [1] Andrew Waterman and Krste Asanović. *The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 20191213*. RISC-V Foundation, 2019.